

In any case, Apache Kafka guarantees that a message is read only once at the group level, irrespective of the number of consumers in the group. This architecture gives consistency, high-performance, high scalability, near-real-time delivery, and message persistence along with zero-message loss.

Installing and running Kafka

Although, in theory, the Apache Kafka cluster can consist of any number of brokers, most of the clusters in production environments usually consist of three or five of these.

Here, we will set up a single-broker cluster that is good enough for the development environment.

Download the latest version of Kafka from <https://kafka.apache.org/downloads> using a browser. It can also be downloaded with the following command, on a Linux terminal:

```
wget https://www.apache.org/dyn/closer.cgi?path=/kafka/2.8.0/kafka_2.12-2.8.0.tgz
```

We can move the downloaded archive file `kafka_2.12-2.8.0.tgz` to some other folder, if needed. Extracting the archive creates a folder by the name `kafka_2.12-2.8.0`, which will be referred to as `KAFKA_HOME` hereafter.

Open the file `server.properties` under the `KAFKA_HOME/config` folder and uncomment the line with the following entry:

```
listeners=PLAINTEXT://:9092
```

This configuration enables Apache Kafka to receive plain text messages on port 9092, on the local machine. Kafka can also be configured to receive messages over a secure channel as well, which is recommended in the production environments.

Irrespective of the number of brokers, Apache Zookeeper is required for broker management and coordination. This is true even in the case of single-broker clusters. Since Zookeeper is already bundled with Kafka, we can start it with the following command from `KAFKA_HOME`, on a terminal:

```
./bin/zookeeper-server-start.sh ./config/zookeeper.properties
```

Once Zookeeper starts running, Kafka can be started in another terminal, with the following command:

```
./bin/kafka-server-start.sh ./config/server.properties
```

With this, a single-broker Kafka cluster is up and running.

Verifying Kafka

Let us publish and receive messages on the topic `topic-1`. A topic can be created with a chosen number of partitions with the following command:

```
./bin/kafka-topics.sh --create --topic topic-1 --zookeeper localhost:2181 --partitions 3 --replication-factor 1
```

The above command also specifies the replication factor, which should be less than or equal to the number of brokers in the cluster. Since we are working on a single-broker cluster, the replication factor is set to one.

Once the topic is created, producers and consumers can exchange messages on that topic. The Kafka distribution includes a producer and a consumer for test purposes. Both of these are command-line tools.

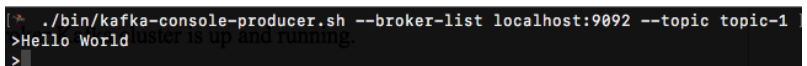
To invoke the producer, open the third terminal and run the following command:

```
./bin/kafka-console-producer.sh --broker-list localhost:9092 --topic topic-1
```

This command displays a prompt at which we can key in simple text messages. Because of the given options on the command, the producer sends the messages on `topic-1` to the Kafka that is running on port 9092 on the local machine.

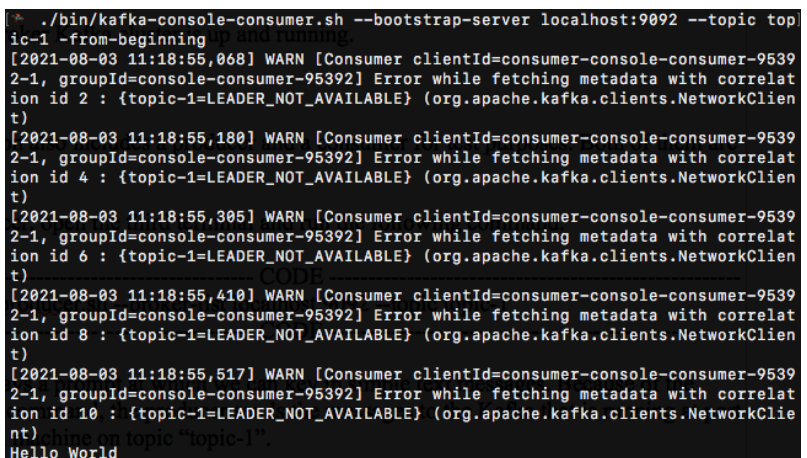
Open the fourth terminal and run the following command to start the consumer tool:

```
./bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic topic-1 --from-beginning
```



```
./bin/kafka-console-producer.sh --broker-list localhost:9092 --topic topic-1
>Hello World
>
```

Figure 3: Command-line producer



```
./bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic topic-1 --from-beginning
[2021-08-03 11:18:55,068] WARN [Consumer clientId=consumer-console-consumer-95392-1, groupId=console-consumer-95392] Error while fetching metadata with correlation id 2 : {topic-1=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient)
[2021-08-03 11:18:55,180] WARN [Consumer clientId=consumer-console-consumer-95392-1, groupId=console-consumer-95392] Error while fetching metadata with correlation id 4 : {topic-1=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient)
[2021-08-03 11:18:55,305] WARN [Consumer clientId=consumer-console-consumer-95392-1, groupId=console-consumer-95392] Error while fetching metadata with correlation id 6 : {topic-1=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient)
[2021-08-03 11:18:55,410] WARN [Consumer clientId=consumer-console-consumer-95392-1, groupId=console-consumer-95392] Error while fetching metadata with correlation id 8 : {topic-1=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient)
[2021-08-03 11:18:55,517] WARN [Consumer clientId=consumer-console-consumer-95392-1, groupId=console-consumer-95392] Error while fetching metadata with correlation id 10 : {topic-1=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient)
Hello World
```

Figure 4: Command-line consumer